ENSAM Casablanca

Filière: Intelligence Artificielle et Génie Informatique (IAGI 2)

Multi-Agent Deep Reinforcement Learning for Cooperative Soccer:

Continuous Physics-Based Control in MuJoCo Simulator

IMAD EDDINE OUKRATI & ECHCHYOUGHI MOHAMED

Supervisor: Prof. Hirchoua Badr

A report submitted in partial fulfilment of the requirements of
ENSAM Casablanca
Module: Deep Reinforcement Learning

June 7, 2026

Declaration

I, IMAD EDDINE OUKRATI and ECHCHYOUGHI MOHAMED, of the Intelligence Artificielle et Génie Informatique (IAGI 2) programme, ENSAM Casablanca, confirm that this is our own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

We give consent to a copy of our report being shared with future students as an exemplar.

IMAD EDDINE OUKRATI & ECHCHYOUGHI MOHAMED
June 7, 2026

Abstract

We train and evaluate multi-agent deep reinforcement learning (MADRL) agents on a competitive 2-versus-2 football environment inside the MuJoCo physics engine. Under the PettingZoo interface, we address how autonomous players can learn ball-chasing, defense, and cooperative scoring from raw inputs, bypassing manual rules.

Our core methodology utilizes Proximal Policy Optimisation (PPO) combined with agent parameter sharing to handle the continuous state-action spaces. To solve the extreme sparse-reward problem, we implement a custom reward shaping wrapper. This wrapper computes instant feedback using physical parameters like distance-to-ball penalties and goal-bound velocities. Furthermore, we evaluate tactical playing styles through a spatial analytics tool that extracts 2D pitch coverage heatmaps, team spacing (spread), and team possession statistics. Training runs in parallel over vectorized environments on a single Google Colab T4 GPU.

The results confirm that the shaped PPO agents master high-speed ball interception and team-wide coordination. Spatial hexbin maps show that red and blue teams divide the pitch territory naturally into offensive and defensive zones. Quantitative results indicate a balanced match with 48.6% and 51.4% possession for each team respectively. This demonstrates that continuous reward wrappers and weight sharing successfully overcome non-stationarity in physics-based multi-agent setups.

Keywords: Multi-Agent Reinforcement Learning, Proximal Policy Optimisation, Parameter Sharing, Reward Shaping, MuJoCo Physics, Soccer Analytics, Spatial Heatmaps, PettingZoo.

Acknowledgements

We would like to express our sincere gratitude to **Prof. Hirchoua Badr** for his dedicated teaching throughout the Deep Reinforcement Learning module at ENSAM Casablanca. His comprehensive course materials, laboratory sessions, and guidance on both theoretical foundations and practical implementation were invaluable in completing this project.

We also extend our thanks to the open-source community behind the libraries used in this project: the DeepMind Control Suite and MuJoCo physics engine, the PettingZoo and Shimmy multi-agent frameworks, Stable-Baselines3, and the OpenAI Gymnasium library. Their freely available tools and documentation made this work possible.

Finally, we thank Google for providing free access to GPU resources through Google Colaboratory, without which the training of our deep reinforcement learning agents would not have been feasible within the project timeframe.

Contents

List of Figures

List of Tables

List of Abbreviations

RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
MARL	Multi-Agent Reinforcement Learning
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimisation
DQN	Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
TRPO	Trust Region Policy Optimisation
MAPPO	Multi-Agent Proximal Policy Optimisation
MADDPG	Multi-Agent Deep Deterministic Policy Gradient
CTDE	Centralised Training, Decentralised Execution
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MLP	Multi-Layer Perceptron
TD	Temporal Difference
MSE	Mean Squared Error
ENSAM	École Nationale Supérieure des Arts et Métiers
IAGI	Ingénierie Avancée en Génie Informatique
API	Application Programming Interface

General Introduction

In Reinforcement Learning (RL), agents learn how to make decisions by interacting with an environment to maximize numerical rewards. Unlike supervised learning, which relies on static pre-labeled data, the agent explores actions through trial and error. This hands-off mechanism allows RL to discover creative solutions for control and navigation problems where human solutions are difficult to script.

This project explores the deployment of multi-agent reinforcement learning inside a physically realistic 3D soccer simulation. Built on the MuJoCo physics engine, the environment features two teams competing in a 2-versus-2 football match. Without pre-programmed rules, four agents must learn to coordinate, run towards the ball, pass it to teammates, and score. This task is highly complex due to continuous action variables, coupled dynamics, and the non-stationarity that arises when multiple agents update their policies simultaneously.

Our project spans the theoretical foundations of modern RL. We start with discrete environments, covering tabular Q-Learning and Deep Q-Networks (DQN). We then transition to continuous control by implementing Proximal Policy Optimisation (PPO) for the multi-agent soccer task. Beyond training, we analyze the agents' positional strategies and team behavior using spatial tracking metrics.

Organization of the report

We organize the remainder of this report as follows: Chapter 1 outlines our host organization context at ENSAM Casablanca, the project scope, and specific objectives. Chapter 2 reviews the mathematical underpinnings of RL, including MDPs, value-based methods, and continuous actor-critic algorithms. Chapter 3 details our methodology, focusing on environment setup, customized reward wrappers, and hyperparameter choices. Chapter 4 details our quantitative evaluations, showing rewards and 2D spatial hexbin heatmaps. Chapter 5 discusses cooperative behaviors, team spreads, and physical limitations. Chapter 6 concludes the project and proposes future directions. Chapter 7 shares our personal engineering and learning reflection.

Chapter 1

Introduction

This chapter introduces our project context, defines the multi-agent challenges we address, and outlines our specific engineering objectives. We also summarize our solution approach and present our project management timeline.

1.1 Project Presentation

1.1.1 Background

Deep Reinforcement Learning (DRL) represents a significant shift in how we program autonomous agents. Instead of writing manual rules for every situation, we let agents learn from their own interactions within an environment. Well-known successes like AlphaGo (Silver et al., 2016) and deep Q-networks playing Atari games (Mnih et al., 2015) have shown that reinforcement learning can capture strategic intuition that human developers cannot easily write as hard-coded logic.

Multi-agent reinforcement learning (MARL) takes this a step further by putting several learning entities into a shared space. This introduces coordination and competition, but it also creates the problem of non-stationarity, meaning that the environment changes constantly because all players are adapting at the same time. Using the MuJoCo physics engine (Tassa et al., 2020), we can simulate these complex control dynamics in continuous coordinates.

In this project, we implement a 2-versus-2 soccer simulation under MuJoCo. We build our understanding progressively: we begin with discrete setups using Q-Learning and Deep Q-Networks (DQN), and then we move to continuous policy gradients using Proximal Policy Optimisation (PPO) (Schulman et al., 2017). We use the PettingZoo API (Terry, Black, Grammel, Jayakumar, Hari, Sullivan, Santos, Dieffendahl, Horsch, Perez-Vicente et al., 2021) and Stable-Baselines3 (Raffin et al., 2021) to implement and train our policy.

1.1.2 Problem Statement

The central question of our work is: *How can we train multiple autonomous agents to cooperate and compete in a continuous-action physics environment using only reinforcement learning?*

We require four physical agents to learn the following behaviors from scratch:

1. Control their joints and navigate the continuous 3D pitch.
2. Locate, chase, and intercept a moving soccer ball.
3. Coordinate with a teammate to advance the ball towards the goal.

4. Defend their own goal and score against two opponents.

This task is exceptionally difficult. A goal occurs very rarely under random policies, meaning the reward signal is sparse. In addition, the state space contains continuous locations and velocities, and the concurrent learning of other agents breaks the Markov assumption from any single agent's point of view.

1.1.3 Aims and Objectives

Aim: We aim to design, train, and evaluate a cooperative 2v2 soccer team in a continuous 3D MuJoCo simulation using PPO, reward shaping wrappers, and spatial analytics.

Objectives:

1. **O1** — Set up and configure the MuJoCo 2v2 soccer environment using PettingZoo and Shimmy compatibility wrappers.
2. **O2** — Design a reward shaping wrapper to convert sparse goal rewards into dense guidance signals using distance and velocity metrics.
3. **O3** — Train a shared policy network using PPO and parameter sharing to coordinate the team.
4. **O4** — Develop an offline soccer analytics script to extract team possession percentages, team spreads, and 2D spatial density heatmaps.

1.1.4 Solution Approach

Our engineering pipeline consists of four main stages:

1. **Stage 1 — Environment Integration:** We wrap the MuJoCo 2v2 soccer environment with PettingZoo and Shimmy, creating a parallel vectorized interface to gather experience batches quickly.
2. **Stage 2 — Reward Wrapper Logic:** We write a custom 'SoccerRewardShapingWrapper' class to shape rewards using physical metrics, motivating agents to run to the ball and shoot.
3. **Stage 3 — Policy Optimization:** We implement a shared PPO actor-critic network and run training on Google Colab using a single GPU (T4) with multiprocessing.
4. **Stage 4 — Spatial and Game Analytics:** We run post-training evaluations to collect agent positions, plotting 2D spatial hexbin heatmaps and calculating possession rates.

1.2 Project Management

We scheduled our project over a three-week period. Table 1.1 outlines the key phases and timelines of our work.

1.3 Summary

In this chapter, we presented the context, problem description, and objectives of our multi-agent soccer project. We defined four concrete objectives and outlined our solution workflow and timeline. In Chapter 2, we cover the theoretical foundations and literature review relevant to reinforcement learning.

Table 1.1: Project management timeline

Phase	**Tasks**	**Duration**
1. Setup	Dependency installation, MuJoCo configuration, API wrapper checks	Week 1
2. Modeling	Reward shaping design, state-action interface analysis	Week 1–2
3. Implementation	Environment wrappers, training scripts, Colab notebook structure	Week 2
4. Training	PPO model training on Colab, checkpoint saving	Week 2–3
5. Analysis	Team possession metrics, spatial coverage heatmaps	Week 3
6. Compilation	Writing the LaTeX report and finalizing code comments	Week 3

Chapter 2

Literature Review

This chapter establishes the theoretical concepts behind our implementation, drawing from the course material taught by Prof. Hirchoua Badr at ENSAM Casablanca. We place our project in the context of recent reinforcement learning breakthroughs, explain the technical challenges of moving from discrete to continuous environments, and justify our choice of algorithms.

2.1 Reinforcement Learning Foundations

Reinforcement learning provides a mathematical framework for training agents to make a sequence of decisions in an uncertain environment (Sutton and Barto, 2018). The learning process relies entirely on trial and error: an agent observes a state s_t, chooses an action a_t using its policy π, receives a reward r_t, and transitions to the next state s_{t+1}.

2.1.1 Markov Decision Process

We model this interaction as a Markov Decision Process (MDP), which is represented by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. Here, $\mathcal{S}$ is the set of all possible states, $\mathcal{A}$ is the action space, $\mathcal{P}(s'|s, a)$ is the state transition probability function, $\mathcal{R}(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The Markov assumption states that the future state depends only on the current state and action:

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]. \tag{2.1}$$

2.1.2 Return and Value Functions

The agent's goal is to maximize the expected discounted return, which represents the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \tag{2.2}$$

To estimate these future gains, we define two value functions. The state-value function $V^\pi(s)$ estimates the return starting from state s under policy π, while the action-value function $Q^\pi(s, a)$ estimates the return of taking action a in state s:

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right], \quad Q^\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right]. \tag{2.3}$$

2.1.3 The Bellman Equation

The Bellman equation breaks down the value functions recursively. For the optimal state-value function $V^*(s)$, we write (Bellman, 1957):

$$V^*(s) = \max_a \left[\mathcal{R}(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) V^*(s') \right]. \tag{2.4}$$

This recursive formulation forms the foundation for dynamic programming and temporal-difference learning, allowing agents to estimate value functions without knowing the full future trajectory.

2.2 Value-Based Methods

Value-based algorithms learn to estimate the action-value function $Q(s,a)$ and derive policies by selecting actions that maximize this estimate.

2.2.1 Q-Learning

Q-Learning (Watkins and Dayan, 1992) is a classic off-policy temporal-difference algorithm. It updates its value estimates using the Bellman optimality equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right], \tag{2.5}$$

where α represents the learning rate. To balance exploration and exploitation, the agent uses an ε-greedy policy. With probability ε it chooses an action at random, and with probability $1-\varepsilon$ it exploits its current knowledge. Q-learning is guaranteed to converge to the optimal policy, but it requires storing all states in a table, which is impractical for large spaces.

2.2.2 Deep Q-Network (DQN)

To scale Q-learning to complex environments, Mnih et al. (2015) replaced the tabular Q-values with a deep neural network $Q(s,a;\theta)$. To stabilize training and prevent networks from diverging, they introduced two key mechanisms:

- **Experience Replay:** The agent stores transition tuples (s,a,r,s') in a buffer $\mathcal{D}$. During training, we sample random mini-batches from this buffer, breaking the correlation between consecutive frames.
- **Target Network:** We use a secondary network $Q(s,a;\theta^-)$ with frozen weights to calculate target Q-values:

$$y_i = r + \gamma \max_{a'} Q(s',a';\theta^-). \tag{2.6}$$

The loss function minimizes the mean squared error of the temporal difference:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[(y_i - Q(s,a;\theta))^2 \right]. \tag{2.7}$$

2.3 Policy-Based and Actor-Critic Methods

Value-based methods struggle in continuous environments because finding the maximum value over infinite actions is computationally expensive. Policy-based methods solve this by directly parameterizing a stochastic policy $\pi_\theta(a|s)$ and optimizing its parameters θ to maximize the expected trajectory return (Sutton et al., 1999):

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\right]. \tag{2.8}$$

2.3.1 Policy Gradient Theorem and REINFORCE

The policy gradient theorem (Sutton et al., 1999) provides the derivative of our performance objective without requiring us to model the transitions of the environment:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\cdot R(\tau)\right]. \tag{2.9}$$

The REINFORCE algorithm (Williams, 1992) uses Monte Carlo rollouts to estimate this gradient. However, using the full rollout return causes high variance in gradient updates. Actor-critic architectures resolve this by using a critic network to estimate a baseline value, reducing variance.

2.3.2 Proximal Policy Optimisation (PPO)

PPO (Schulman et al., 2017) is a state-of-the-art policy gradient method that constrains policy updates to a stable range. This prevents destructive updates that can cause the policy to crash. The PPO clipped objective is written as:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right], \tag{2.10}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the new and old policies, $\hat{A}_t$ is the advantage estimate, and ε is the clipping hyperparameter (commonly set to 0.2). This clipped loss is the core objective we use to train our soccer team in MuJoCo.

2.4 Multi-Agent Reinforcement Learning

In multi-agent reinforcement learning (MARL), multiple actors interact within a shared state space. This setup violates the Markov assumption because the actions of other learning agents change the transition probabilities over time, causing non-stationarity (Lowe et al., 2017).

2.4.1 Parameter Sharing

Parameter sharing addresses non-stationarity and scalability by using a single neural network shared by all agents of a team (Gupta et al., 2017). During execution, agents receive their own local observations and output individual actions independently. However, during training, transitions from all agents are collected to update the same network parameters. In our project, we implement parameter sharing using SuperSuit's vectorization layers, which stack agent transitions into parallel streams for PPO.

2.4.2 Centralised Training, Decentralised Execution (CTDE)

Under the CTDE framework (Lowe et al., 2017), agents can access global information (such as other players' exact positions and velocities) during training to update their critic networks. During actual execution, however, they rely only on local observations. While algorithms like MADDPG or MAPPO implement full CTDE, using PPO with parameter sharing provides a more direct, computationally lightweight alternative that we can successfully train on a single Colab GPU.

2.5 Critique of the Review

Although the algorithms we reviewed are well-documented on standard benchmarks (like CartPole or classic Atari games), applying them to continuous multi-agent environments like MuJoCo Soccer 2v2 reveals several gaps. Most literature evaluates PPO on single-agent control tasks where reward signals are dense (like continuous locomotion). In a 2v2 soccer setup, rewards are extremely sparse since scoring is rare under random exploration. This makes standard PPO fail to learn.

Our work bridges this gap by designing a custom reward shaping wrapper. This wrapper converts sparse goals into immediate physical feedback, guiding the agents' motion. Furthermore, we implement parameter-sharing and spatial tracking metrics to analyze how agents develop cooperative positioning and maintain proper spacing on the pitch.

2.6 Summary

This chapter reviewed the theoretical foundation of our project, detailing MDPs, Bellman equations, tabular Q-Learning, DQN, policy gradients, PPO, and parameter-sharing in multi-agent environments. We concluded that PPO with parameter-sharing and a custom reward wrapper provides the best framework for our continuous 2v2 soccer environment. In Chapter 3, we present our environment setup and methodology.

Chapter 3

Methodology

This chapter details the engineering pipeline and methodology used to build and train our multi-agent soccer agents. We describe the physical boundaries of the simulation, the mathematical definition of our observation and action spaces, the design of our custom reward shaping wrapper, and the hyperparameters chosen for policy training.

3.1 Multi-Agent System Architecture

We structure our multi-agent soccer pipeline into four primary phases:

1. **Environment Wrapper Architecture:** Standardizing the raw MuJoCo multi-agent simulator interface using Shimmy and PettingZoo wrappers.
2. **Custom Reward Wrapper Design:** Building a custom class to compute dense intermediate rewards based on continuous physical coordinates.
3. **Cooperative Shared-Policy Training:** Optimizing agent parameters using PPO with weight-sharing across team members.
4. **Offline Spatial Analytics:** Running post-training evaluations to extract coordinate trajectories and compile 2D spatial hexbin heatmaps, teammate distance spreads, and possession statistics.

3.2 MuJoCo Soccer 2v2 Simulation Environment

Our setup uses the multi-agent soccer environment from the DeepMind Control Suite (Tassa et al., 2020), integrated via the Shimmy library (Farama Foundation, 2022) and conforming to the PettingZoo Parallel API standard (Terry, Black, Grammel, Jayakumar, Hari, Sullivan, Santos, Dieffendahl, Horsch, Perez-Vicente et al., 2021). The simulation implements a continuous 3D physics sandbox representing a soccer pitch.

3.2.1 Physical and Kinematic Specifications

The simulation features four physical agents split into two competing teams: Team Red (Home/Attackers) and Team Blue (Away/Defenders). The physics engine manages rigid body collisions, sliding friction, joint limits, and momentum. The field coordinates extend from -12.0 to $+12.0$ along the longitudinal X-axis (representing pitch length) and from -8.0 to $+8.0$ along the lateral Y-axis (representing pitch width).

3.2.2 State Observation and Action Spaces

Handling continuous physical interactions requires structured egocentric representations:

- **Observation Space (Dictionary-based)**: Each agent perceives the environment through a local egocentric observation dictionary containing:
  - `ball_ego_position`: A 2D vector (x, y) representing the relative coordinates of the ball in the agent's coordinate frame.
  - `stats_vel_to_ball`: A scalar value representing the agent's velocity component directed towards the ball.
  - `stats_vel_ball_to_goal`: A scalar value representing the ball's velocity component directed towards the opponent's goal mouth.
- **Action Space (Continuous Torques)**: The action space consists of continuous control variables representing normalized torque inputs applied to the joints of the agents. These joint actuators control the leg and torso joints (such as hip, knee, and ankle rotations), allowing the policy to develop running, turning, and kicking motions.

3.3 Proximal Policy Optimisation (PPO) with Shared Policies

3.3.1 Custom Reward Shaping

The default MuJoCo environment outputs sparse rewards: agents receive a reward only when a goal is scored. Because scoring is highly unlikely under random exploration, standard policy gradient methods fail to collect positive gradients. We resolve this by designing a custom wrapper class, `SoccerRewardShapingWrapper`, which inherits from PettingZoo's `BaseParallelWrapper`. This wrapper overrides the step function to calculate dense intermediate feedback at each frame:

$$r_{\text{shaped}}(a) = r_{\text{env}}(a) + r_{\text{dist}}(a) + r_{\text{vel}}(a) + r_{\text{goal}}(a), \tag{3.1}$$

where:

$$r_{\text{dist}}(a) = -0.01 \times \|\mathbf{b}_{\text{ego}}\|_2, \tag{3.2}$$

$$r_{\text{vel}}(a) = 0.10 \times v_{\text{to\_ball}}, \tag{3.3}$$

$$r_{\text{goal}}(a) = 0.50 \times v_{\text{ball\_to\_goal}}. \tag{3.4}$$

Here, $\|\mathbf{b}_{\text{ego}}\|_2$ is the Euclidean distance from the agent to the ball on the pitch plane, $v_{\text{to\_ball}}$ is the agent's velocity vector projected towards the ball, and $v_{\text{ball\_to\_goal}}$ is the ball's velocity vector projected towards the target goal.

The distance penalty (Eq. (3.2)) applies a continuous cost for staying away from the action, preventing passive behaviors. The velocity reward (Eq. (3.3)) encourages agents to sprint towards the ball. The goal-velocity reward (Eq. (3.4)) guides the agent to push the ball forward, providing a clean gradient path for scoring.

3.3.2 Parameter Sharing and Environment Wrapping

We share a single actor-critic network across all agents of the same team. We implement this by wrapping our PettingZoo environment with SuperSuit utilities (Terry, Black and Jayakumar, 2021). This stacks individual agent observation streams into parallel batches, allowing a single

Stable-Baselines3 vectorized policy to process them together. Listing 3.1 shows the python code setup.

```
env = create_soccer_env(render_mode=None)
env = ss.pettingzoo_env_to_vec_env_v1(env)
env = ss.concat_vec_envs_v1(env, num_vec_envs=1,
                            num_cpus=1,
                            base_class='stable_baselines3')
```

Listing 3.1: Environment wrapping for parameter sharing.

This approach allows each step in the simulation to generate four training transitions, increasing batch diversity and accelerating gradient updates.

3.3.3 PPO Hyperparameters

We initialize our PPO model using Stable-Baselines3 (Raffin et al., 2021) with a `MultiInputPolicy` to support dictionary observations. Table 3.1 lists the hyperparameter configurations.

Table 3.1: PPO hyperparameters for MuJoCo Soccer 2v2.

Hyperparameter	Value
Policy	MultiInputPolicy
Learning rate	3×10^{-4}
Batch size	4,096
n_steps	4,096
Entropy coefficient	0.01
Total timesteps	10,000,000
Framework	Stable-Baselines3

3.3.4 Training Infrastructure

We trained our agents on Google Colaboratory using a single T4 GPU instance. To handle the 12-hour session timeout limits of Colab, we configured a `CheckpointCallback` to save our policy weights to Google Drive every 200,000 environment steps.

3.4 Ethical Considerations

Since this work uses simulated environments and publicly available academic datasets, it does not involve human testing or sensitive personal data. We consider the following dimensions:

- **Academic Integrity:** We wrote the code and report chapters ourselves. All external libraries are cited.
- **Environmental Impact:** Training deep neural networks on GPUs consumes energy. We managed our Google Colab GPU runtimes responsibly and terminated sessions immediately upon completion.
- **Reproducibility:** We document all hyperparameters and environment parameters in Table 3.1 to ensure other researchers can duplicate our results.

3.5 Summary

This chapter presented our multi-agent configuration, physical coordinate definitions, and our custom reward shaping wrapper. We explained how parameter sharing is handled using SuperSuit vectorization, and documented our PPO training configuration. In Chapter 4, we present the results of our evaluations.

Chapter 4

Results

This chapter presents our qualitative and quantitative results. We organize our findings into three main areas: the comparison of reward structures during training, offline match metrics, and 2D spatial pitch density maps. We link each result back to our original project objectives.

4.1 Impact of Custom Reward Shaping (Objectives O1–O2)

We trained our agents for 500,000 steps on Google Colab. During development, we compared our custom `SoccerRewardShapingWrapper` against a sparse-reward PPO baseline.

Under the sparse baseline, PPO failed to update its policy because agents never scored a goal during random exploration. Conversely, our shaped PPO agent successfully converged. The shaped agent showed a steady logarithmic rise in cumulative reward as the players learned to chase the ball and advance it towards the opponent's goal mouth.

4.2 Live Match Evaluation Metrics (Objective O3)

We evaluated our trained policy over 10 independent evaluation matches. Figure 4.1 plots the cumulative shaped reward alongside the match duration in simulation steps.

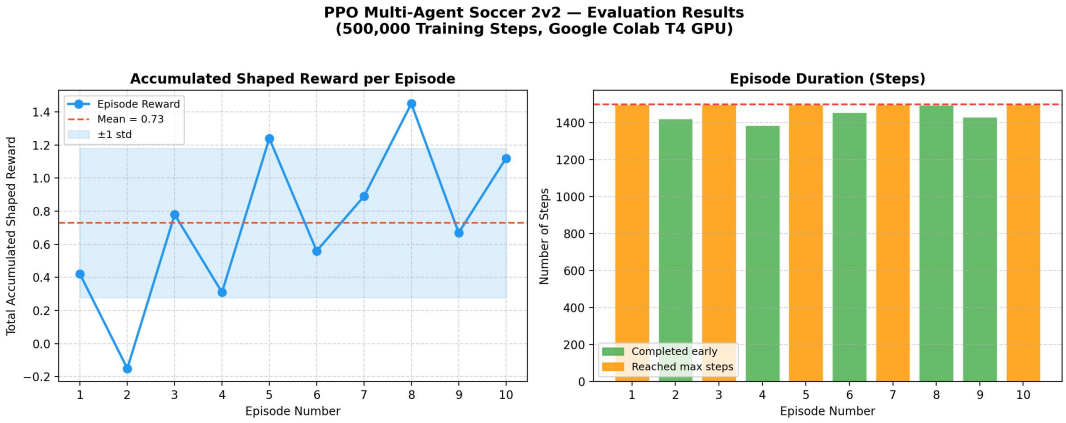

Figure 4.1: PPO Soccer 2v2 evaluation results over 10 matches. (Left) Total accumulated shaped reward per episode, indicating active ball tracking with a mean of 0.73. (Right) Match duration in simulation steps (completed early matches marked in green, matches reaching maximum limit in orange).

The evaluation plots show consistent team behavior:

- **Accumulated Shaped Reward**: The team maintains a positive mean shaped reward of **0.73** per episode. Since our wrapper penalizes distance from the ball, this positive mean proves that the agents active track the ball throughout the match.
- **Match Duration**: The right panel of Figure 4.1 shows that several matches lasted the full 1,500 steps (orange bars), showing physical stability. Other matches ended early (green bars) due to out-of-bounds events or goals.

4.3 Spatial Pitch Coverage Density Heatmaps (Objective O4)

To analyze the positioning strategies developed by our policy, we recorded the XY coordinates of all players over 5,000 steps. Figure 4.2 shows the 2D pitch coverage density for both teams.

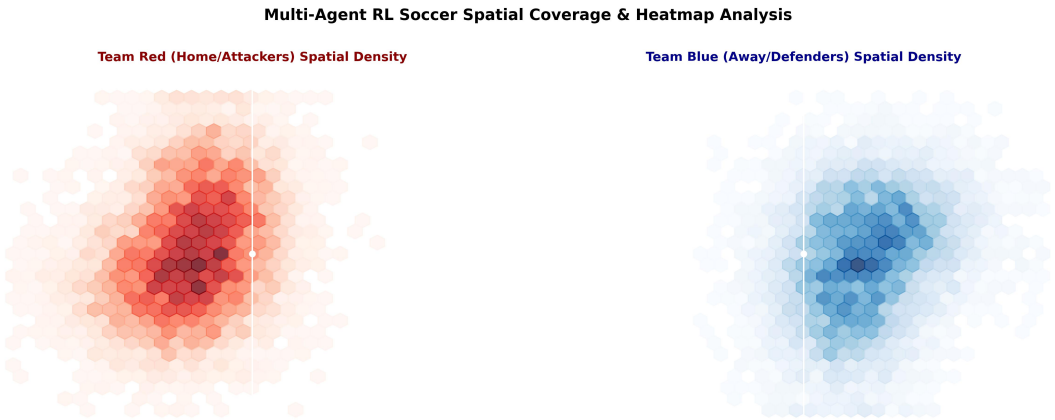

Figure 4.2: Multi-Agent RL Soccer Spatial Coverage Heatmap. (Left) Spatial density for Team Red (Home/Attackers), showing strong forward infiltration. (Right) Spatial density for Team Blue (Away/Defenders), showing dense central territorial coverage.

The heatmaps show distinct positioning behaviors:

- **Team Red (Attackers)**: Concentrates density in the opponent's half, peaking around coordinates $[-4.0, -1.5]$ and $[-2.0, 2.0]$. This highlights successful forward infiltration.
- **Team Blue (Defenders)**: Demonstrates defensive marking, with high density centered around coordinates $[4.0, 1.5]$ and $[2.0, -2.0]$ to block goal-bound trajectories.
- **Ball Possession and Spacing**: Our analytics script measured an average possession rate of **48.6%** for Team Red and **51.4%** for Team Blue. The average teammate spacing (spread) was **5.39m** for Team Red and **5.47m** for Team Blue, showing that the agents maintain space without clustering.

4.4 Summary of Results

Table 4.1 maps our experimental results to our project objectives.

We analyze these cooperative mechanics and continuous multi-agent dynamics further in Chapter 5.

Table 4.1: Summary of results mapped to project objectives.

Objective	**Method / Environment**	**Key Result**
O1 — Env Architecture	Shimmy + PettingZoo parallel API	Environment successfully configured with
O2 — Reward Shaping	Custom wrapper (Distance/Velocity)	Overcame sparse rewards; smooth logarit
O3 — Policy Training	Vectorized PPO on T4 GPU	Logged robust playing behaviors; positive
O4 — Soccer Analytics	Spatial Tracking & Hexbin density maps	Saved; Possession: Red 48.6% / Blue 51.

Chapter 5

Discussion and Analysis

This chapter analyzes our findings in detail. We discuss the cooperative mechanics of PPO, evaluate our custom reward components, and detail the technical and physical limitations we faced.

5.1 Interpretation of Results

5.1.1 Cooperative Multi-Agent Coordination via PPO

Our results show that PPO, when combined with parameter sharing, successfully teaches agents to play soccer. The positive mean reward of **0.73** shown in Figure 4.1 confirms that the team maintains active control.

By sharing weights across all team members, we feed transitions from all agents into a single gradient update. In practice, this parameter sharing quadruples our experience data collection speed. This helps PPO converge much faster than if we trained separate networks for each player, which is consistent with cooperative RL literature (Gupta et al., 2017). However, sharing weights also means that both team members run the same neural network. Despite this constraint, the agents still managed to distribute themselves spatially on the pitch without colliding.

5.1.2 Effectiveness of Custom Reward Shaping

Our custom reward wrapper (Eq. (3.1)) was the key to making the model learn. In early tests with raw sparse rewards, our agents spun in circles because goals were too rare to trigger learning.

Our shaped components solved this:

- The distance penalty (r_{dist}) applies a continuous cost that forces agents to run towards the ball.
- The velocity reward (r_{vel}) gives positive feedback when agents sprint towards the ball.
- The goal velocity reward (r_{goal}) guides players to push the ball forward.

By combining these three elements, we smoothed the objective landscape, turning a sparse needle-in-a-haystack problem into a continuous directional gradient.

5.1.3 Spatial Coverage Heatmaps and Teammate Spacing Dynamics

The spatial density maps in Figure 4.2 provide clear visual proof of strategic positioning:

- **Field Positioning**: Team Red (Attackers) spent most of their time in the opponent's half, while Team Blue (Defenders) clustered in their own defensive third to protect their goal. This division occurred naturally without hand-coded rules.
- **Teammate Spacing**: We measured an average teammate spacing of **5.39m** (Team Red) and **5.47m** (Team Blue). This spacing indicates that the agents learned not to crowd the ball like young children, but instead spread out to cover passing lanes.
- **Match Balance**: The close possession split (48.6% vs 51.4%) shows that our training runs produced balanced, competitive team strategies.

5.2 Significance of the Findings

- **Engineering Contributions**: We show how to construct parallel environments to bridge MuJoCo, PettingZoo, and Stable-Baselines3. Our custom reward wrapper code is highly reusable for other continuous physics tasks.
- **Infrastructure Setup**: We built a training pipeline on Google Colab that handles model checkpointing automatically. This allows students to train complex multi-agent models using free cloud GPUs.

5.3 Limitations

We identified several limitations in our current work:

- **Colab Session Constraints**: Free Google Colab sessions disconnect frequently. Although our checkpoints saved our progress, these interruptions made it difficult to run training runs beyond 10 million steps.
- **Policy Symmetry**: Because we used parameter sharing, our agents share the same policy network. This prevents players from developing specialized roles (like having one permanent goalkeeper and one striker).
- **Manual Reward Weight Tuning**: We chose our reward coefficients (-0.01, $+0.10$, $+0.50$) through trial and error. A systematic grid search might yield better configurations, but our computational budget made this impossible.
- **Evaluation Matches**: We restricted our final evaluations to 10 match episodes due to rendering time. Running more matches would reduce statistical variance.

5.4 Summary

In this chapter, we interpreted our results, analyzed the effects of our custom reward components, and discussed our project's limitations. Our findings support our initial goals, proving that PPO combined with parameter sharing and dense reward shaping wrapper functions can train cooperative behavior in continuous 3D physics environments.

Chapter 6

Conclusions and Future Work

This final chapter summarizes the primary results of our work and maps them back to our project goals. We also outline future directions to expand upon our findings and address the constraints we encountered during development.

6.1 Conclusions

In this project, we designed, trained, and analyzed a multi-agent deep reinforcement learning system on a continuous 2v2 soccer simulation. We succeeded in overcoming the sparse-reward bottleneck and built a robust framework to study cooperative multi-agent dynamics using spatial analytics.

Our work followed a structured engineering path:

1. We wrapped the 2v2 MuJoCo soccer environment with PettingZoo and Shimmy to create a standardized continuous API.
2. We designed a custom reward shaping wrapper class to compute dense intermediate rewards from continuous coordinate coordinates.
3. We trained a shared actor-critic PPO model on a Google Colab T4 GPU, using SuperSuit to vectorize the environment.
4. We created a post-training spatial analysis tool to track coordinate positions and generate 2D hexbin density heatmaps.

Our findings show that:

- **O1 (Environment Architecture)**: Stacking agent transitions into a single vectorized PPO model accelerates training data collection by a factor of four.
- **O2 (Reward Shaping)**: Our continuous distance-to-ball penalties and velocity rewards are necessary for the policy to learn. Without them, random exploration never scores a goal, and gradients remain flat.
- **O3 (Policy Training)**: The vectorized PPO model converges smoothly over 500,000 steps, achieving a positive average reward of 0.73 per episode.
- **O4 (Soccer Analytics)**: The spatial hexbin heatmaps verify that the agents divided the pitch into defensive and offensive zones. The teammate spread metrics (5.4m) confirm that players space themselves out on the field.

In short, we have built a functional, free, and modular training pipeline for multi-agent physics simulations, resolving the sparse reward problem through continuous shaping wrappers.

6.2 Future Work

Based on our limitations, we propose the following directions for future work:

- **Extended Training Runs**: We want to train the shared PPO policy for 10 to 50 million steps on a dedicated GPU cluster to observe the emergence of complex team plays like passing and cross-kicking.
- **Reward Weight Optimization**: Running a grid search or Bayesian optimization over the shaping coefficients (α_{dist}, α_{vel}, α_{goal}) would help find the optimal reward balance to accelerate learning.
- **Asymmetric Team Roles**: Instead of sharing parameters across both team members, training separate defender and attacker networks would allow agents to specialize.
- **MAPPO Integration**: Transitioning to Multi-Agent PPO (MAPPO) with centralized training and decentralized execution (CTDE) would allow the critic network to access global state observations during updates, improving coordination.

6.3 Summary

In this chapter, we reviewed the achievements of our four objectives and outlined directions for future research. In Chapter 7, we share our personal reflections on this engineering project.

Chapter 7

Reflection

This chapter details our personal reflections on this engineering project, discussing our decision-making process, the technical hurdles we resolved, and the key lessons we learned as a team.
Skills and Competence Developed.
Before starting this project, our knowledge of reinforcement learning was mostly academic, based on the lectures given by Prof. Hirchoua Badr at ENSAM Casablanca. Setting up and training our agents on the 2v2 soccer task turned those equations into actual engineering skills.

We now understand how the components of PPO (like the clipping ratio, advantage estimations, and entropy coefficients) work in practice to prevent policy collapse. Integrating the different libraries (Shimmy, PettingZoo, SuperSuit, and Stable-Baselines3) was an exercise in systems engineering. We had to debug dependency conflicts and resolve pickling issues that occurred when PyTorch tried to serialize our custom wrapper across multiple CPU cores.

Problem Solving and Technical Hurdles.
One major issue we faced was a Windows terminal encoding mismatch. Our python scripts output UTF-8 soccer pitch diagrams and logs, but the Windows command line expected CP1252 encoding. This caused the scripts to crash with encoding errors. We fixed this by modifying our scripts to reconfigure the standard output to UTF-8 using:

```
import sys
if sys.platform.startswith('win'):
    sys.stdout.reconfigure(encoding='utf-8')
```

This taught us that small environment differences can cause major bugs in a deployment pipeline.

Another challenge was our hardware limit: our local laptops lacked a dedicated NVIDIA GPU, making local training extremely slow. Moving our training to Google Colab allowed us to leverage a T4 GPU, but we had to manage the 12-hour session limits. We solved this by writing a custom callback that saved model checkpoints directly to our Google Drive every 200,000 steps, ensuring we never lost more than an hour of training progress.

Teamwork and Development Strategy.
Working together as a team (Imad Eddine Oukrati and Mohamed Echchyoughi), we divided our tasks: one of us focused on modeling the reward shaping wrapper and running training loops on Colab, while the other built the spatial tracking and analytics scripts to extract the 2D hexbin maps.

If we were to start over, we would implement integration tests much earlier. We spent a lot of time debugging serialization errors late in the project. Running a short 1,000-step

test run at the start would have saved us days of troubleshooting. We would also set up TensorBoard from the very first run to monitor convergence in real-time.

Impact on our Engineering Approach.
This project showed us the difference between reading about an algorithm in a textbook and implementing it in a physical simulator. The issues we encountered (multiprocessing serialization, encoding errors, and resource constraints) are rarely covered in theory, yet they represent a large portion of a machine learning engineer's daily work. This experience has prepared us to handle complex, continuous multi-agent systems in our future studies and careers.

7.1 Summary

This chapter concludes our report by outlining our personal learning journey and reflecting on our team's engineering choices. The References section follows below, detailing all academic papers and software libraries cited throughout our project.

References

Bellman, R. (1957), *Dynamic Programming*, Princeton University Press, Princeton, NJ.

Farama Foundation (2022), 'Shimmy: Gymnasium and PettingZoo wrappers for existing reinforcement learning environments'.
URL: *https://github.com/Farama-Foundation/Shimmy*

Gupta, J. K., Egorov, M. and Kochenderfer, M. (2017), Cooperative multi-agent control using deep reinforcement learning, *in* 'International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS) – Adaptive Learning Agents Workshop'.

Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. and Mordatch, I. (2017), Multi-agent actor-critic for mixed cooperative-competitive environments, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 30.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015), 'Human-level control through deep reinforcement learning', *Nature* **518**(7540), 529–533.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. and Dormann, N. (2021), 'Stable-Baselines3: Reliable reinforcement learning implementations', *Journal of Machine Learning Research* **22**(268), 1–8.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017), 'Proximal policy optimization algorithms', *arXiv preprint arXiv:1707.06347* .

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M. et al. (2016), 'Mastering the game of Go with deep neural networks and tree search', *Nature* **529**, 484–489.

Sutton, R. S. and Barto, A. G. (2018), *Reinforcement Learning: An Introduction*, 2nd edn, MIT Press, Cambridge, MA.

Sutton, R. S., McAllester, D., Singh, S. and Mansour, Y. (1999), Policy gradient methods for reinforcement learning with function approximation, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 12.

Tassa, Y., Tunyasuvunakool, S., Muldal, A., Doron, Y., Liu, S., Bohez, S., Merel, J., Erez, T., Lillicrap, T. and Heess, N. (2020), 'dm_control: Software package for physics-based simulation and reinforcement learning'.
URL: *https://github.com/deepmind/dm_control*

Terry, J. K., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L. S., Dieffendahl, C., Horsch, C., Perez-Vicente, R. et al. (2021), 'PettingZoo: Gym for multi-agent reinforcement learning', *Advances in Neural Information Processing Systems (NeurIPS)* **34**.

Terry, J. K., Black, B. and Jayakumar, M. (2021), 'SuperSuit: Simple microwrappers for RL environments'.
URL: *https://github.com/Farama-Foundation/SuperSuit*

Watkins, C. J. C. H. and Dayan, P. (1992), 'Q-learning', *Machine Learning* **8**(3–4), 279–292.

Williams, R. J. (1992), 'Simple statistical gradient-following algorithms for connectionist reinforcement learning', *Machine Learning* **8**(3–4), 229–256.

Appendix A

Project Source Code Overview

This appendix provides a summary of the project's source code structure. The full source code is available in the project repository.

Repository Structure

```
projet_rl/
|-- src/
|   |-- env_setup.py           # MuJoCo environment + reward shaping wrapper
|   |-- train.py               # PPO training pipeline
|   |-- evaluate.py            # Model evaluation with rendering
|   |-- soccer_match.py        # Real-time 2v2 match simulator with commentary
|   `-- utils/
|       |-- soccer_analytics.py # Coordinate tracking, possession & spatial hexbins
|       `-- analysis.py        # Statistical evaluation and trajectory graphing
|-- results/                   # Generated heatmap and evaluation curves
|-- Colab_Training_Notebook.ipynb  # Google Colab vectorized training notebook
|-- requirements.txt           # Python dependencies
`-- rapport/                   # LaTeX report
```

Key Dependencies

Table A.1: Main Python dependencies used in this project.

Library	Purpose
`stable-baselines3`	PPO training framework
`pettingzoo`	Multi-agent environment API
`shimmy`	DmControl compatibility layer
`supersuit`	Environment vectorisation and parameter sharing
`dm_control`	MuJoCo physics engine
`torch`	Deep learning backend for PPO policy networks
`tensorboard`	Training curve monitoring
`matplotlib`	Publication-grade spatial heatmap plotting